

Requirement Analysis

PocketSmart AI – Your Smart Budget & Recommendation Assistant

1. Purpose

Requirement analysis defines what PocketSmart AI should provide to users and what technical components are required to implement the system.

The system must collect user requirements, process budget-related information using AI and generate personalized recommendations.

2. Functional Requirements

2.1 User Registration

The system should allow new users to create an account.

Required information may include:

- Username
 - Email
 - Password
 - User credentials
-

2.2 User Login

Registered users should be able to securely log in.

The system should authenticate users and create a session for personalized interaction.

2.3 User Logout

Users should be able to terminate their current session through the logout functionality.

3. Home Interior Planner Requirements

The Home Interior Planner should allow users to:

- Enter their budget.
- Select room types.
- Specify quantities.
- Select required products.
- Provide style preferences.

The system should generate suitable home interior recommendations.

4. Party Planner Requirements

The Party Planner should collect:

- Total budget
- Guest count
- Event type
- Venue details
- Food requirements
- Decoration requirements

The system should generate suitable party planning recommendations.

5. Jewelry Planner Requirements

The Jewelry Planner should collect:

- Budget
- Occasion
- Style preference
- Outfit details

The system should optionally accept an outfit image.

The AI should use the available information to generate matching jewelry suggestions.

6. Recommendation Requirements

The recommendation engine should:

1. Understand the user's budget.
 2. Understand user preferences.
 3. Identify the selected category.
 4. Generate suitable recommendations.
 5. Consider price and relevance.
 6. Provide platform-specific product or service suggestions.
-

7. Authentication and Session Requirements

The system requires authentication-related functionality including:

- `/register`
- `/login`
- `/logout`
- `/token`
- `/session-info`
- `/session-data`

JWT-based authentication is used for authorized access to protected API routes.

8. History Requirements

Users should be able to access previous recommendation queries and results.

The `/history` functionality supports reviewing and reusing previous planning information.

9. Non-Functional Requirements

Performance

The application should respond to user requests efficiently.

Usability

The interface should be simple and easy to understand.

Security

User authentication and session information should be handled securely.

Scalability

The architecture should allow additional planners and recommendation categories to be added in the future.

Maintainability

The application should use a modular backend structure for easier maintenance.

10. Technology Requirements

Frontend

- HTML
- CSS
- JavaScript
- Jinja2 Templates

Backend

- Python
- FastAPI

AI

- Gemini 1.5 Flash Pro

Server

- Uvicorn

Additional Components

- JWT Authentication
 - Session Management
 - CORS
 - Environment Variables
-

11. Overall Requirement Flow

